

prog2 lab B1

Esercizi TDD

Esercizio 1

- 1) data una funzione C, creare un ambiente per il testing
- 2) il test deve essere richiamato da una funzione apposita nello stesso modulo del testato che restituirà il risultato del test
- 3) input ed output nella versione finale potranno essere stringhe da convertire nel dato richiesto dal test
- 4) il programma finale sarà composto da due files: il main() che può richiamare sia la funzione normale che quella per testarla
- 5) usare `argc > 1` per decidere se fare test o meno

Esercizio 1

Esercizio (file quadrato.c):

→ calcola il quadrato di un numero intero

- 1) partiamo da una funzione semplice richiamata una volta sola
- 2) riempiamo due array: uno per i valori di input ed uno per i valori di output corretti.
- 3) creiamo un ciclo che richiama la funzione con un input dell'array e controlliamo se il risultato coincide con quello previsto
- 4) se no stampiamo un messaggio con errore differenza etc.
- 5) scrivere a tal proposito un programma “TestDriver” che implementa la slide 15 (della serie slide precedente)

Esercizio 1

Seconda iterazione:

terminata la slide precedente,

1) aggiungere un altro test

123456 → -1 // Overflow!

1) riaggiustare il codice (dovremo aggiungere una funzione)

Esercizio 1

per trattare correttamente questo caso

123456 $\rightarrow$ -1

occorre una funzione che inizializzi correttamente il “massimo quadrato senza overflow”.

1. create una variabile e una funzione che nel main() e' richiamata per calcolare il suo valore prima della test suite:

```
int trovamax(int n); // qui potete usare anche la moltiplicazione
```

Esercizio 1

Digressione:

L'overflow e' una brutta bestia, C di per se' non vi da errori quando si verifica, dovete voi evitarlo **prima** che avvenga. occorre quindi una funzione che trovi il limite cosi' da fare un check preventivo sul massimo.

In C tutti i controlli sono sacrificati se impattano la velocita', e' il programmatore ad aggiungerli

Come trovare Overflow con un **int** ?

facciamo due passi:

Esercizio 1.1

- 1) scrivete una funzione trovamax() che a tentativi trovi l' **int** piu' grande che al quadrato non da overflow (pseudocodice):

```
curr = 1
```

```
while not overflow :
```

```
    res = curr*curr
```

```
    if checkoverflow(res):
```

```
        overflow = true
```

```
    else:
```

```
        curr = curr + 1
```

```
return curr-1
```

Esercizio 1.1

2) come trovare overflow?

e' undefined! dipende dalla CPU!

sul mio computer un po' di test ha detto (per un **int**):

se $X > 0$ e $X * X < 0 \rightarrow \text{Overflow}$

Ma non e' sempre vero!!!! (ho scritto un esempio):

Allora ho aggiunto un altro test (piu' generico) se :

$X * Y / Y \neq X \rightarrow \text{Overflow}$

cioe' se moltiplico e divido per la stessa quantita' devo tornare al valore di partenza

Esercizio 1.2

- per i test anziche' input output con tipo "int" usate il tipo stringa (char*)
- leggete una stringa da un "array di char" in input , convertitela in int, chiamate quadrato() convertite il risultato in stringa e confrontatelo coll' elemento di test_out
- quindi dovete definire due funzioni come in una precedente slide:

```
char * to_string(int x);  
int from_string(char *s);
```

- potete assumere che non ci sia error checking (piu' o meno ..?)

Esercizio 1.2

Perche' questo lavoro aggiuntivo?

- Perche' a volte i valori di input/output sono memorizzati su file.
- il test_driver legge i file, converte i dati ed esegue i test.
- In genere modificare i file e' meglio che modificare il codice
- Ovviamente la conversione da/a stringhe puo' essere dispendiosa da realizzare nel caso di I/O molto strutturato, con i mezzi che abbiamo a disposizione in questo corso.

Esercizio 1.2

- Se il programma produce dei messaggi di errore (es Overflow), come facciamo a rilevarli?
- la faccenda diventa piu' laboriosa. Esempio:
- ```
if (x > MASSIMO_VALORESENZA_OVERFLOW){
```
- ```
    fprintf(stderr,"warning overflow detected %d\n",x);
```
- ```
}
```
- ```
.....
```
- il messaggio corretto viene stampato in output ma la test suite non lo rileva

Test Driven Development

Occorre allora eseguire in una shell separata il programma, recuperare il file stdout/stderr, e confrontarli con quelli corretti

→ questi tipi di test spesso sono eseguiti da un altro test_driver:

```
for filein, fileout in coppie_testfile:
    print("testing", file ,fileout)
    risultato = run_executable(filein)
    if different(fileout,risultato) :
        print("fail",risultato)
    else:
        print("pass")
```

Questo e' solo uno schema, ogni programma ha le sue peculiarita'.

Esercizio 1.3

Estensione / Ottimizzazione:

Siccome siamo in C, dopo che abbiamo fatto **trovamax()** sequenziale, perche' non farla piu' veloce? allora sfruttiamo la ricerca **binaria**. dividiamo ogni volta l'intervallo a meta' e in $\log_2(N)$ tentativi terminiamo

provate a vedere i tempi confrontando le due versioni.

Esercizio 1.4

vi divertite a programmare ? Altra estensione all'esercizio:

- partendo dalla funzione quadrato, **mantenete** l'uso della somma ed estendetela progressivamente a :
 int potenza(int x, int esp); dove $0 \leq \text{esp} \leq N$
- in particolare: se $x < 0$ ed esp e' dispari, il segno del risultato sara' negativo, attenti sempre ai casi limite (0,-1,1,MAX,-MAX..) sia per la base che l'esponente.
- aggiungete i test case opportuni all'elenco. notate che adesso ogni test ha 2 input , quindi occorrera' un array per ogni parametro dei test
- estendete anche la costante MAXSQUARE che vale solo per il caso 2 ad un array che valga per tutti gli esponenti ed inizializzatelo al primo richiamo di potenza(x,y) (notate che bastano meno di 100 posti, visto che INT_MAX di solito e' $2^{63} - 1$)

Esercizio 1.4

suggerimento: data la potenza in base 2 , quella in base 3 si ottiene sommando x volte quella in base 2 e cosi' via... (ricorsivamente o meno) aggiungete alla coppia textin, texout, un terzo array char *testcomment[] che stamperete nel caso di fallimento del relativo test

se non lo avete ancora fatto, riorganizzate il codice in 3 sorgenti:

- 1) il main.c richiama le funzioni di test()
- 2) le funzioni di test sono in test.c
- 3) le funzioni matematiche sono in <nomechevoglio.c>

occorrono anche i 2 .h con i prototipi delle relative funzioni

Esercizio 1.4

riassunto dell'esercizio: scrivere una funzione

```
int potenza(int x, int esp);  
int e' un qualunque intero;  
esp e' un int >= 0;
```

la funzione stampa un messaggio di errore OVERFLOW su stderr
in caso opportuno, altrimenti funziona correttamente per tutti gli altri casi

sviluppare in modo

- iterativo,
- incrementale,
- tdd